SNOWFLAKE SQL PRACTICAL

Subject: Data Foundations

Platform: Snowflake Cloud Data Warehouse

Topic: Database Creation, Warehouse Management, Table Creation, CSV Data Loading and SQL Operations

Objective:

To understand and practise Snowflake SQL operations by creating a database, managing schemas and virtual warehouses, creating tables, preparing CSV file formats, uploading files to internal stages, and loading and verifying data using SQL commands.

PART 1: DATABASE AND SESSION MANAGEMENT

Step 1: Check Current Session Information

Objective: To retrieve information about the current Snowflake session.

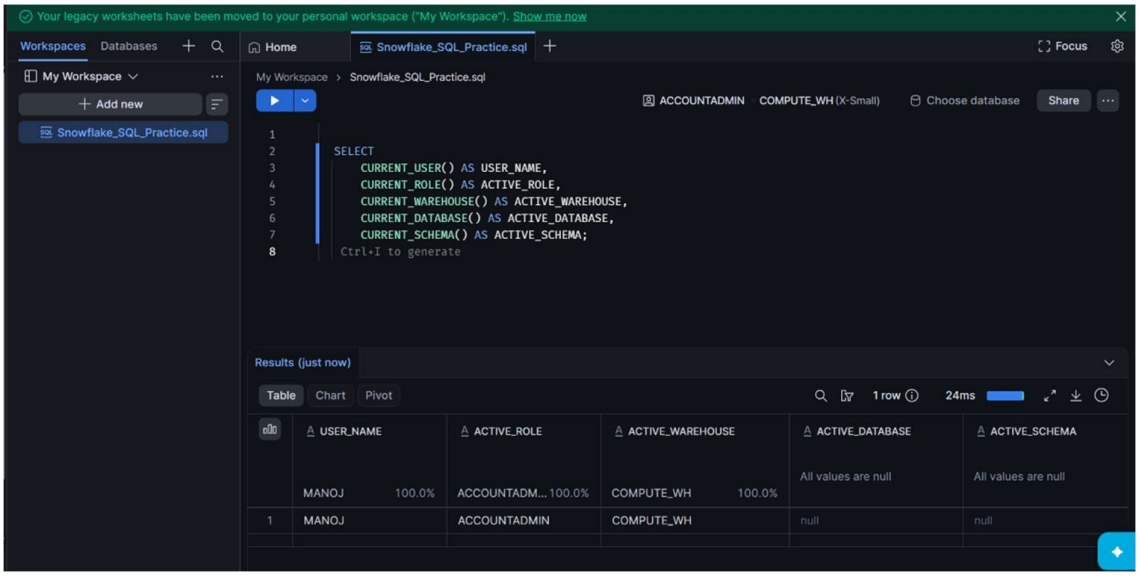
SQL Command:

```
SELECT
  CURRENT_USER() AS USER_NAME,
  CURRENT_ROLE() AS ACTIVE_ROLE,
  CURRENT_WAREHOUSE() AS ACTIVE_WAREHOUSE,
  CURRENT_DATABASE() AS ACTIVE_DATABASE,
  CURRENT_SCHEMA() AS ACTIVE_SCHEMA;
```

Explanation:

- CURRENT_USER() displays the logged-in user.
- CURRENT_ROLE() displays the active role.
- CURRENT_WAREHOUSE() displays the active virtual warehouse.
- CURRENT_DATABASE() displays the selected database.
- CURRENT_SCHEMA() displays the selected schema.

Observation: The query displays the current Snowflake session details and helps identify the active working environment.



Step 2: Create a Database

Objective: To create a new database in Snowflake for storing company-related information.

SQL Command:

CREATE DATABASE IF NOT EXISTS COMPANY_DATA;

Explanation:

- CREATE DATABASE creates a new database.
- IF NOT EXISTS avoids errors if the database already exists.
- COMPANY_DATA is the name of the database.

Verification Command:

SHOW DATABASES LIKE 'COMPANY_DATA';

Observation: The database was created successfully and verified using the SHOW DATABASES command.



Step 3: Create a Schema

Objective: To create a schema inside the COMPANY_DATA database for organizing employee-related database objects.

SQL Command:

CREATE SCHEMA IF NOT EXISTS COMPANY_DATA.HR_RECORDS;

Explanation:

- CREATE SCHEMA creates a new schema.
- IF NOT EXISTS prevents an error if the schema already exists.
- COMPANY_DATA is the database name.
- HR_RECORDS is the schema name.

Select the Schema:

USE SCHEMA COMPANY_DATA.HR_RECORDS;

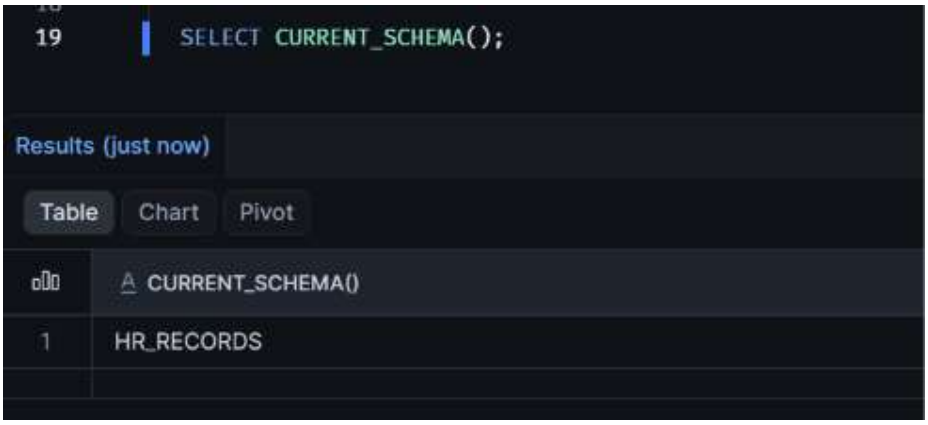
This command selects HR_RECORDS as the active schema.

Verification Command:

SHOW SCHEMAS IN DATABASE COMPANY_DATA;

SELECT CURRENT_SCHEMA();

Observation: The HR_RECORDS schema was created inside the COMPANY_DATA database and selected successfully for subsequent operations.



PART 2: VIRTUAL WAREHOUSE MANAGEMENT

Step 4: Create a Virtual Warehouse

Objective: To create a virtual warehouse in Snowflake that provides the computing resources required to execute SQL queries.

SQL Command:

CREATE WAREHOUSE IF NOT EXISTS STAFF_WH

WAREHOUSE_SIZE = 'XSMALL'

AUTO_SUSPEND = 60

AUTO_RESUME = TRUE

INITIALLY_SUSPENDED = TRUE;

Explanation:

- CREATE WAREHOUSE creates a virtual warehouse.
- IF NOT EXISTS prevents an error if the warehouse already exists.
- STAFF_WH is the warehouse name.
- WAREHOUSE_SIZE = 'XSMALL' specifies the smallest standard warehouse size.
- AUTO_SUSPEND = 60 automatically suspends the warehouse after 60 seconds of inactivity.
- AUTO_RESUME = TRUE allows the warehouse to resume automatically when required.
- INITIALLY_SUSPENDED = TRUE creates the warehouse in a suspended state.

Select the Warehouse:

USE WAREHOUSE STAFF_WH;

This command selects STAFF_WH as the active virtual warehouse.

Verification Commands:

SHOW WAREHOUSES LIKE 'STAFF_WH';

SELECT CURRENT_WAREHOUSE();

Observation: The STAFF_WH virtual warehouse was created with an XSMALL configuration and automatic suspension and resumption settings. It was selected as the active warehouse for subsequent SQL operations.





PART 3: TABLE CREATION

Step 5: Create an Employee Table

Objective: To create an employee details table inside the COMPANY_DATA database and HR_RECORDS schema.

SQL Command:

CREATE TABLE IF NOT EXISTS

COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS

```
(
   EMPLOYEE_ID NUMBER(10,0),
   EMPLOYEE_NAME VARCHAR(100),
   DEPARTMENT VARCHAR(50),
   SALARY NUMBER(12,2),
   LOCATION VARCHAR(100)
);
```

Explanation:

- CREATE TABLE creates a new table.
- IF NOT EXISTS prevents an error if the table already exists.
- EMPLOYEE_ID stores the employee identification number.
- EMPLOYEE_NAME stores the employee name.
- DEPARTMENT stores the employee's department.
- SALARY stores the employee's salary.
- LOCATION stores the employee's location.

Table Structure:

Column	Data Type	Purpose
EMPLOYEE_ID	NUMBER(10,0)	Employee identifier
EMPLOYEE_NAME	VARCHAR(100)	Employee name
DEPARTMENT	VARCHAR(50)	Department
SALARY	NUMBER(12,2)	Employee salary
LOCATION	VARCHAR(100)	Employee location

Verification Command:

```
SHOW TABLES LIKE 'EMPLOYEE_DETAILS'
IN SCHEMA COMPANY_DATA.HR_RECORDS;
```

Check Table Structure:

```
DESCRIBE TABLE COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS;
```

Observation: The EMPLOYEE_DETAILS table was created successfully with five columns and their respective data types.

```
36  CREATE TABLE IF NOT EXISTS
37  COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS
38  (
39      EMPLOYEE_ID NUMBER(10,0),
40      EMPLOYEE_NAME VARCHAR(100),
41      DEPARTMENT VARCHAR(50),
42      SALARY NUMBER(12,2),
43      LOCATION VARCHAR(100)
44  );
```

Results (just now)

Table | Chart | Pivot

	status
1	Table EMPLOYEE_DETAILS successfully created.

```
51
52  DESCRIBE TABLE
53  COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS;
54  Ctrl+I to generate
```

Results (just now)

Table | Chart | Pivot — 5 rows — 36ms

	name	type	kind	null?	default	primary key	unique key	chec
1	EMPLOYEE_ID	NUMBER(10,0)	COLUMN	Y	null	N	N	null
2	EMPLOYEE_NAME	VARCHAR(100)	COLUMN	Y	null	N	N	null
3	DEPARTMENT	VARCHAR(50)	COLUMN	Y	null	N	N	null
4	SALARY	NUMBER(12,2)	COLUMN	Y	null	N	N	null
5	LOCATION	VARCHAR(100)	COLUMN	Y	null	N	N	null

PART 4: FILE FORMAT CREATION

Step 6: Create a CSV File Format

Objective: To create a CSV file format that defines how Snowflake reads and interprets CSV files during data loading.

SQL Command:

CREATE FILE FORMAT IF NOT EXISTS

COMPANY_DATA.HR_RECORDS.STAFF_CSV_FORMAT

TYPE = CSV

SKIP_HEADER = 1

FIELD_OPTIONALLY_ENCLOSED_BY = '"'

EMPTY_FIELD_AS_NULL = TRUE;

Explanation:

- CREATE FILE FORMAT creates a named file format.
- STAFF_CSV_FORMAT is the file format name.
- TYPE = CSV specifies the CSV file type.
- SKIP_HEADER = 1 skips the first row containing column names.
- FIELD_OPTIONALLY_ENCLOSED_BY = '"' handles values enclosed in double quotes.
- EMPTY_FIELD_AS_NULL = TRUE converts empty fields into NULL values.

Verification Command:

SHOW FILE FORMATS

IN SCHEMA COMPANY_DATA.HR_RECORDS;

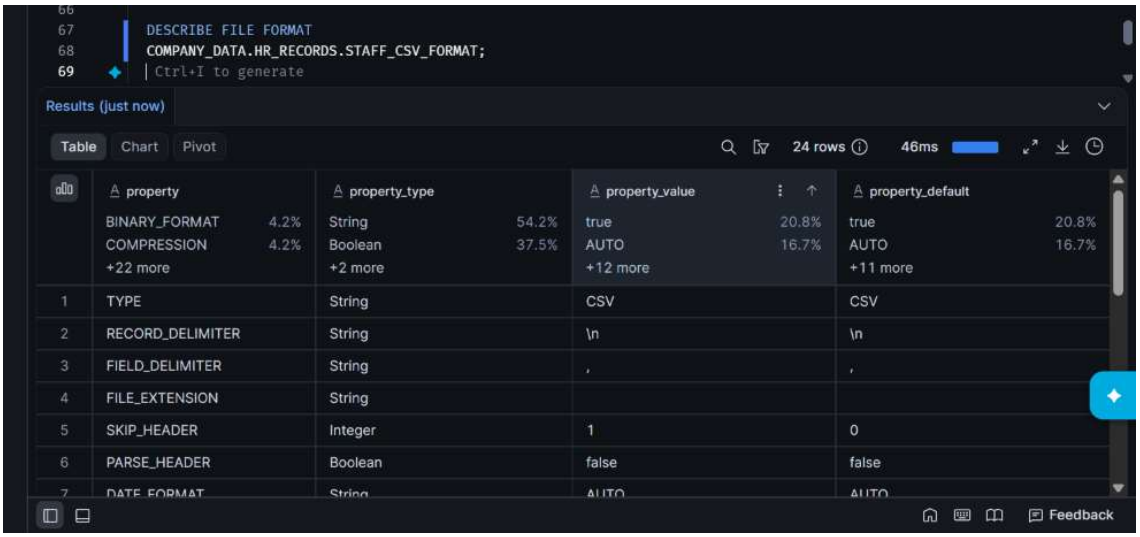
View File Format Properties:

DESCRIBE FILE FORMAT

COMPANY_DATA.HR_RECORDS.STAFF_CSV_FORMAT;

Observation: The STAFF_CSV_FORMAT file format was created successfully with CSV parsing settings. It is configured to skip the header row, handle double-quoted fields, and interpret empty fields as NULL values.





PART 5: INTERNAL STAGE MANAGEMENT

Step 7: Create an Internal Stage

Objective: To create an internal stage in Snowflake for storing CSV files before loading them into database tables.

SQL Command:

CREATE STAGE IF NOT EXISTS

COMPANY_DATA.HR_RECORDS.STAFF_STAGE

FILE_FORMAT = COMPANY_DATA.HR_RECORDS.STAFF_CSV_FORMAT;

Explanation:

- CREATE STAGE creates a named stage in Snowflake.
- IF NOT EXISTS prevents an error if the stage already exists.
- STAFF_STAGE is the name of the internal stage.
- FILE_FORMAT specifies the CSV file format associated with the stage.
- STAFF_CSV_FORMAT is the previously created file format.

Verification Command:

SHOW STAGES

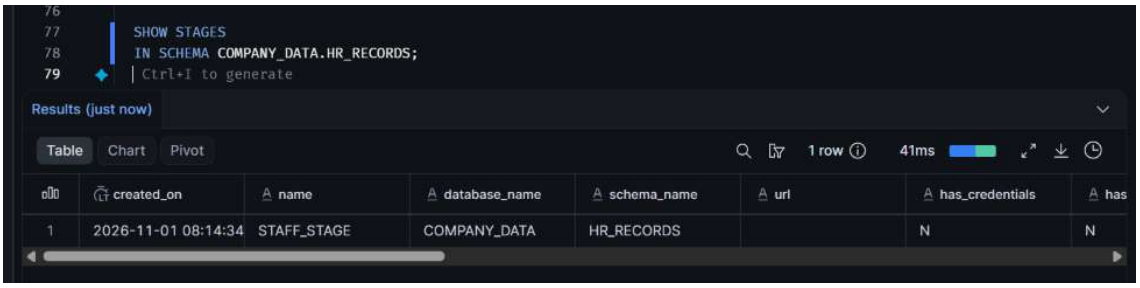
IN SCHEMA COMPANY_DATA.HR_RECORDS;

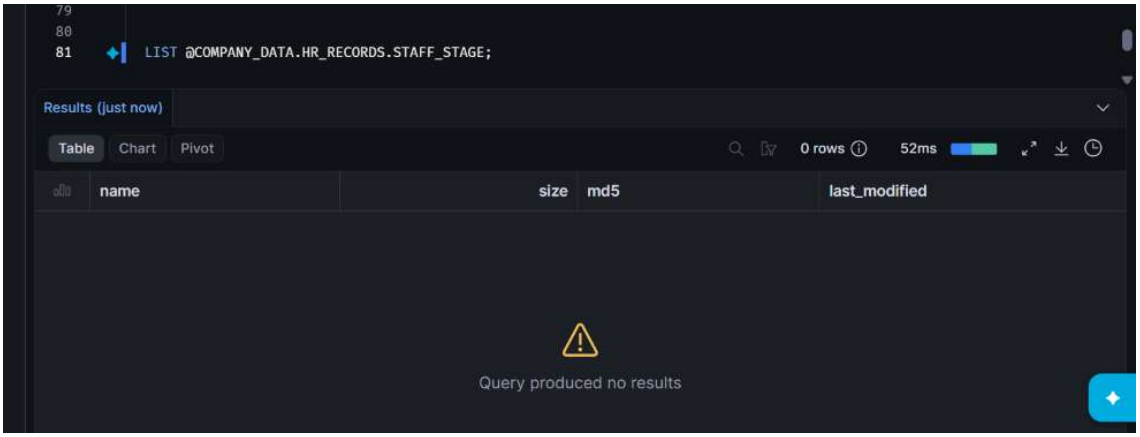
List Files Inside the Stage:

LIST @COMPANY_DATA.HR_RECORDS.STAFF_STAGE;

Observation: The STAFF_STAGE internal stage was created successfully and associated with the STAFF_CSV_FORMAT file format. The stage is ready to receive employee CSV files.







PART 6: FILE PREPARATION AND DATA UPLOAD

Step 8: Prepare and Upload the CSV File

Objective: To prepare an employee CSV file and upload it into the Snowflake internal stage.

File Name: staff_records.csv

Dataset Description: The CSV file contains eight employee records with the following columns:

- EMPLOYEE_ID
- EMPLOYEE_NAME

- DEPARTMENT
- SALARY
- LOCATION

Data Preparation: The employee records were prepared in CSV format using Notepad and saved as staff_records.csv.

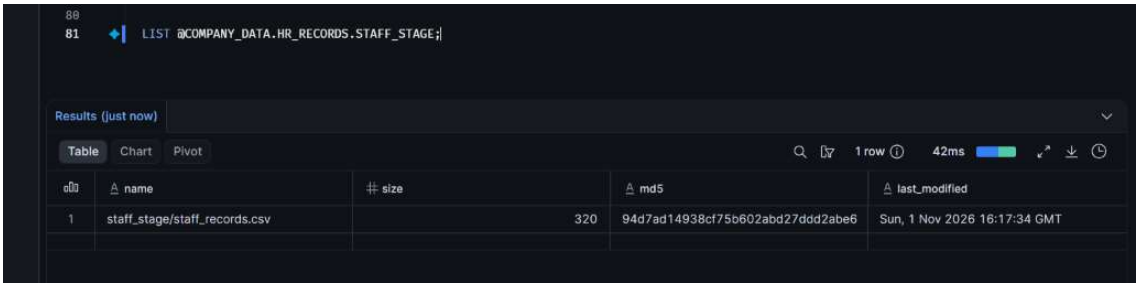
Upload Procedure:

1. Open the Snowflake Snowsight interface.
2. Navigate to the COMPANY_DATA database.
3. Open the HR_RECORDS schema.
4. Select the STAFF_STAGE internal stage.
5. Use the Upload Files option to select and upload staff_records.csv.

Verification Command:

```
LIST @COMPANY_DATA.HR_RECORDS.STAFF_STAGE;
```

Observation: The employee CSV file was uploaded to the STAFF_STAGE internal stage and verified using the LIST command.



PART 7: LOADING DATA INTO SNOWFLAKE

Step 9: Load CSV Data Using COPY INTO

Objective:

To load employee records from the CSV file uploaded to the internal stage into the EMPLOYEE_DETAILS table using Snowflake's COPY INTO command.

SQL Command:

```
COPY INTO COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS
FROM @COMPANY_DATA.HR_RECORDS.STAFF_STAGE
FILE_FORMAT = (
  FORMAT_NAME =
  'COMPANY_DATA.HR_RECORDS.STAFF_CSV_FORMAT'
```

)

ON_ERROR = 'ABORT_STATEMENT';

Explanation:

1. COPY INTO loads data from a staged CSV file into a Snowflake table.
2. The destination table is COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS.
3. The FROM clause specifies the internal stage containing the CSV file.
4. FILE_FORMAT specifies the previously created CSV file format.
5. ON_ERROR = 'ABORT_STATEMENT' stops the loading operation if an error occurs.

Execution Result:

Property	Result
File status	LOADED
Rows parsed	8
Rows loaded	8
Errors seen	0

Observation:

The CSV file was successfully loaded into the EMPLOYEE_DETAILS table. Snowflake parsed all 8 employee records and loaded all 8 rows without encountering any errors.

Screenshot:



PART 8: DATA VERIFICATION

Step 10: Verify the Loaded Employee Records

Objective:

To verify the employee records loaded into the EMPLOYEE_DETAILS table and confirm the total number of records.

Query 1: Display All Records

```
SELECT *
FROM COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS;
```

Query 2: Count Total Employees

```
SELECT COUNT(*) AS TOTAL_EMPLOYEES
FROM COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS;
```

Query 3: Display the First Five Records

```
SELECT *
FROM COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS
LIMIT 5;
```

Explanation:

1. SELECT * retrieves all columns and records from the employee table.
2. COUNT(*) calculates the total number of records in the table.
3. LIMIT 5 restricts the query output to the first five records.

Observation:

The employee records were successfully retrieved from the EMPLOYEE_DETAILS table. The COUNT(*) query confirms that 8 employee records are present in the table.

Result:

The CSV data was successfully loaded and verified in Snowflake.

Screenshots:



	EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT	SALARY	LOCATION
1	2001	Anil	IT	58000.00	Chennai
2	2002	Meena	HR	46000.00	Bangalore
3	2003	Rajesh	Finance	62000.00	Hyderabad
4	2004	Divya	IT	67000.00	Chennai
5	2005	Karthik	Sales	42000.00	Mumbai
6	2006	Priya	HR	49000.00	Hyderabad
7	2007	Suresh	Finance	71000.00	Bangalore
8	2008	Nisha	IT	63000.00	Mumbai

```
104
105   SELECT COUNT(*) AS TOTAL_EMPLOYEES
106   FROM COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS;
```

Results (just now)

Table | Chart | Pivot

	# TOTAL_EMPLOYEES
1	8

```
108   SELECT *
109   FROM COMPANY_DATA.HR_RECORDS.EMPLOYEE_DETAILS
110   LIMIT 5;
```

Results (1 minute ago)

Table | Chart | Pivot — 5 rows — 57ms

	# EMPLOYEE_ID	A EMPLOYEE_NAME	A DEPARTMENT	# SALARY	A LOCATION
1	2001	Anil	IT	58000.00	Chennai
2	2002	Meena	HR	46000.00	Bangalore
3	2003	Rajesh	Finance	62000.00	Hyderabad
4	2004	Divya	IT	67000.00	Chennai
5	2005	Karthik	Sales	42000.00	Mumbai